

Architectural Pattern Selection & Component Modeling Justification

Student: Subramani B M | SRN: PES1UG24CS473 | Section: H | Scenario: Self-Service Coffee Kiosk System

Architecture Selection: “We chose Layered / Client-Server Architecture for the Self-Service Coffee Kiosk System.”

1. Architectural Choice

We evaluated three architectural styles for the standalone Coffee Kiosk terminal: *Microservices Architecture* was deemed excessively complex and resource-heavy for a single dedicated hardware terminal. **Layered Architecture** was selected because it cleanly segments the kiosk software into distinct operational tiers: Touchscreen Presentation, Order Management Logic, Hardware Integration Adapters, and Data Persistence. A *Client-Server* link provides optional centralized pricing synchronization.

2. Two Specific Scenario-Related Reasons for Selection

- a. Deterministic Hardware Coordination & Transactional Workflow:** The kiosk coordinates user touch inputs (Espresso, Americano, Latte; Small, Large sizes) with strict hardware state transitions: order configuration → card reader PIN authorization → thermal receipt printing. The layered pattern enforces deterministic sequencing, ensuring the receipt printer and coffee dispense commands are triggered only upon validated payment confirmation.
- b. Standalone Offline Resilience & Low Resource Footprint:** Café kiosks must maintain high availability during peak hours regardless of internet fluctuations. Embedding a local datastore (*Menu & Pricing DB*) within the data layer enables the kiosk to operate autonomously, process orders, compute totals, and queue transactions without depending on distributed network availability.

3. Security Advantage

Encapsulated PCI-DSS Payment Isolation & Tokenized Card Processing: Because the system accepts credit cards only, the *Payment Service Component* completely isolates communication with the EMV chip/contactless card reader hardware over an encrypted serial link. The presentation, order management, and database components never handle or store raw card numbers (PANs) or CVVs, exchanging only opaque tokens and ensuring strict PCI-DSS compliance.

4. Performance Benefit

Direct In-Memory Bus Execution & Sub-Second Latency: Because all components execute within the kiosk local application runtime, communications across *IOrderService*, *IPrinterDriver*, and *IMenuData* occur via direct in-memory calls and local device drivers (ESC/POS protocol). This eliminates network latency, delivering instantaneous touchscreen feedback (< 50 ms) and rapid receipt printing to maximize customer throughput.

5. Component & Interface Specification Summary

Component Name	Provided Interface (Ball)	Required Interface (Socket)	Core Architectural Responsibility
Touchscreen UI	None (Customer Touch Screen)	IOrderService, IMenuData	Presents drink menu & sizes, captures touch inputs, displays total
Order Manager	IOrderService (placeOrder, calcTotal)	IPaymentProcessor, IPrinterDriver	Manages order lifecycle, calculates prices/taxes, triggers payment/print
Payment Service	IPaymentProcessor (authCard, charge)	Card Reader USB Serial Driver	Interfaces with EMV reader, enforces credit card only, returns token
Printer Controller	IPrinterDriver (printReceipt)	Thermal Printer ESC/POS Driver	Formats itemized receipt, monitors paper status, triggers thermal print
Menu & Pricing DB	IMenuData (getMenu, getPrices)	SQLite In-Memory Engine	Stores drink recipes (Espresso, Latte, Americano), sizes, and pricing